

Advanced MLOps Practice Labs

Introduction to MLOps • ML Lifecycle • Environment Setup • Model
Serialization & Configuration Management

Four advanced, hands-on labs covering production-grade MLOps foundations: maturity-model architecture, reproducible pipeline orchestration, locked and containerized environments, and robust model/config management.

Prerequisites	Completion of basic MLOps/ML engineering labs; comfortable with Python, Docker, Git, and the command line
Tools Used	Docker, conda-lock/pip-tools, MLflow, DVC, Hydra/OmegaConf, ONNX, scikit-learn, GitHub Actions, Great Expectations
Duration	Approx. 1.5 - 2.5 hours per lab

Table of Contents

Lab 1	MLOps Maturity & Architecture Design	3
Lab 2	Orchestrating a Reproducible ML Lifecycle	6
Lab 3	Locked, Containerized Environment Setup	9
Lab 4	Model Serialization & Configuration Management	12

LAB 1

MLOps Maturity & Architecture Design

Assessing maturity levels and designing an end-to-end MLOps reference architecture

Objective	Move beyond ad hoc ML scripts by evaluating an organization against a formal MLOps maturity model and designing a target reference architecture with CI/CD for ML (CI/CD/CT).
Tools	Draw.io / diagrams.py, GitHub Actions, Python 3
Duration	90-120 minutes

1.1 Background

Introductory MLOps treats tooling in isolation (a notebook here, a tracked experiment there). At an advanced level, MLOps is treated as an organizational capability with defined maturity levels — commonly Level 0 (manual, script-driven), Level 1 (ML pipeline automation with continuous training), and Level 2 (full CI/CD/CT automation with automated pipeline deployment). This lab asks you to assess a scenario against this model and design the target architecture, including the often-missed concept of **Continuous Training (CT)** — automatically retraining and redeploying a model when new data or code triggers a pipeline run.

1.2 Scenario

A fraud-detection team currently trains models in local Jupyter notebooks, manually copies the serialized model to a shared drive, and a separate team manually loads it into a Flask service that is redeployed by hand. There is no automated retraining, no data validation, and no rollback mechanism if a new model underperforms.

1.3 Maturity Assessment

Complete the maturity scorecard below for the scenario, then repeat it for your own project/organization:

Capability	Level 0 (Manual)	Level 1 (Pipeline Automation)	Level 2 (CI/CD/CT)
Data validation	None / ad hoc	Automated checks in pipeline	Automated + gated deployment
Training	Manual notebook run	Scripted, parameterized pipeline	Triggered by data/code changes
Deployment	Manual file copy	Scripted deployment	Automated CD with rollback
Monitoring	None	Manual dashboard review	Automated drift/perf alerts + retrain trigger

Rate the fraud-detection scenario against each row (it is Level 0 throughout). Identify which single capability, if upgraded first, would deliver the most risk reduction — justify your choice in 3-4 sentences.

1.4 Design the Target Architecture

Produce an architecture diagram (hand-drawn, draw.io, or the **diagrams** Python library) covering the following components and their data/control flow:

- Source control (feature/training code) with pull-request-triggered CI
- Automated data validation gate (schema, ranges, drift check) before training proceeds
- Pipeline orchestrator (e.g. Kubeflow Pipelines, Airflow, or GitHub Actions matrix jobs) running feature engineering, training, and evaluation as discrete, versioned steps
- Experiment tracking store recording every run's parameters, metrics, and artifacts
- Automated model evaluation gate comparing the candidate against the current production model on a held-out set, blocking promotion if it does not beat a threshold
- Model registry with stage transitions and rollback capability
- CD step that deploys the registry's *Production*-tagged model to a serving endpoint
- Monitoring/observability feeding data and prediction drift signals back into a Continuous Training trigger

1.5 Formalize the CT Trigger

Write pseudocode / a GitHub Actions skeleton for what triggers a retraining pipeline run:

```
# .github/workflows/continuous-training.yml (skeleton)
name: continuous-training
on:
  schedule:
    - cron: '0 3 * * 1'          # weekly scheduled retrain
  repository_dispatch:
    types: [drift-detected]      # triggered externally by a monitoring job

jobs:
  validate-data:
    runs-on: ubuntu-latest
    steps:
      - run: python src/data_validation.py --fail-on-error

  train-and-evaluate:
    needs: validate-data
    runs-on: ubuntu-latest
    steps:
      - run: python src/train.py
      - run: python src/evaluate_against_production.py --min-improvement 0.0

  register-and-promote:
    needs: train-and-evaluate
    if: success()
    runs-on: ubuntu-latest
    steps:
      - run: python src/register_model.py --stage Staging
```

1.6 Exercises

- Extend the workflow with a manual-approval gate before Staging → Production promotion, and justify where in a regulated environment a human-in-the-loop approval should sit.
- Define a rollback procedure: what conditions automatically revert to the previous production model, and how fast should that decision be made?
- Identify two single points of failure in your architecture and propose mitigations.
- Score your own current project against the Level 0/1/2 scorecard and write a one-page roadmap to move it up one level.

#

Viva / Discussion Question

1	What distinguishes Level 1 MLOps maturity from Level 2 in terms of what is automated?
2	Why is an automated evaluation gate (candidate vs. production model) necessary even when training is fully automated?
3	What risk does a fully automatic Continuous Training + Continuous Deployment pipeline introduce in a regulated environment, and how would you mitigate it?
4	How does data drift differ from concept drift, and why does each require a different response?

LAB 2

Orchestrating a Reproducible ML Lifecycle

Building a versioned, reproducible, multi-stage pipeline with data and code lineage

Objective	Implement a full ML lifecycle (ingest -> validate -> feature engineer -> train -> evaluate -> register) as a reproducible, versioned pipeline using DVC and MLflow together.
Tools	Python 3, DVC, MLflow, scikit-learn, Git
Duration	90-120 minutes

2.1 Background

A basic ML lifecycle lab runs one training script end-to-end. An advanced lifecycle treats each stage as an independently versioned, cacheable unit: if only the evaluation code changes, only evaluation should re-run — not the full pipeline. This requires **data versioning** (DVC) layered with **experiment tracking** (MLflow) and a lineage graph connecting a specific model version back to the exact code commit, data version, and parameters that produced it.

2.2 Setup

```
pip install dvc mlflow scikit-learn pandas pyyaml
mkdir mlops_lifecycle_lab && cd mlops_lifecycle_lab
git init
dvc init
mkdir -p src data models metrics
```

2.3 Define a DVC Pipeline (dvc.yaml)

DVC pipelines declare stages, their dependencies, and their outputs, so **dvc repro** only re-executes stages whose inputs actually changed.

```
# dvc.yaml
stages:
  ingest:
    cmd: python src/ingest.py --output data/raw.csv
    outs:
      - data/raw.csv

  validate:
    cmd: python src/validate.py --input data/raw.csv
    deps:
      - data/raw.csv
      - src/validate.py

  featurize:
    cmd: python src/featurize.py --input data/raw.csv --output data/features.csv
    deps:
      - data/raw.csv
      - src/featurize.py
    outs:
      - data/features.csv

  train:
    cmd: python src/train.py --input data/features.csv --model-out models/model.pkl
```

```

deps:
  - data/features.csv
  - src/train.py
params:
  - train.n_estimators
  - train.max_depth
outs:
  - models/model.pkl
metrics:
  - metrics/train_metrics.json:
      cache: false

evaluate:
  cmd: python src/evaluate.py --model models/model.pkl --input data/features.csv
  deps:
    - models/model.pkl
    - src/evaluate.py
  metrics:
    - metrics/eval_metrics.json:
        cache: false

```

2.4 Define Parameters (params.yaml)

```

# params.yaml
train:
  n_estimators: 150
  max_depth: 8
  random_state: 42

```

2.5 Wire MLflow into the Training Stage

Add to **src/train.py** (abbreviated):

```

import json, yaml, mlflow, joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import subprocess

params = yaml.safe_load(open('params.yaml'))['train']
git_sha = subprocess.check_output(['git', 'rev-parse', 'HEAD']).decode().strip()

mlflow.set_experiment('lifecycle-lab')
with mlflow.start_run():
    mlflow.log_params(params)
    mlflow.set_tag('git_sha', git_sha)
    mlflow.set_tag('dvc_stage', 'train')

    df = pd.read_csv('data/features.csv')
    X, y = df.drop(columns=['target'], df['target'])
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=params['random_state'])

    model = RandomForestClassifier(**{k: v for k, v in params.items() if k != 'random_state'},
                                   random_state=params['random_state'])
    model.fit(X_train, y_train)
    acc = accuracy_score(y_val, model.predict(X_val))
    mlflow.log_metric('val_accuracy', acc)
    mlflow.sklearn.log_model(model, 'model')

    joblib.dump(model, 'models/model.pkl')
    json.dump({'val_accuracy': acc}, open('metrics/train_metrics.json', 'w'))

```

2.6 Run and Verify Reproducibility

```
dvc repro
dvc dag          # visualize the pipeline dependency graph
git add dvc.yaml dvc.lock params.yaml src/ .gitignore
git commit -m 'Add reproducible ML lifecycle pipeline'

# Now change ONLY max_depth in params.yaml, then:
dvc repro
# Observe: only 'train' and 'evaluate' stages re-run; 'ingest' and 'featurize' are cached/skip
ped
```

2.7 Reproduce a Past Result

```
git log --oneline          # find an earlier commit
git checkout <commit_sha>
dvc checkout              # restores the exact data/model versions for that commit
dvc repro                 # confirms the pipeline is fully reproducible from that poi
nt
```

2.8 Exercises

- Add a **drift-check** stage between **validate** and **featurize** that compares the new raw data's feature distributions against a stored reference and fails the pipeline if drift exceeds a threshold.
- Use **dvc metrics diff** to compare **val_accuracy** between two Git commits without manually re-running training.
- Add a remote DVC storage backend (local dir, S3, or GCS) and push data/model artifacts with **dvc push**.
- Extend **train.py** to log the DVC pipeline stage hash as an MLflow tag, fully linking an MLflow run back to a specific **dvc.lock** state.

#	Viva / Discussion Question
1	How does DVC's dependency graph decide which pipeline stages need to re-run?
2	Why is it insufficient to version only the code (Git) without also versioning the data and pipeline parameters?
3	What is the difference between a DVC 'metric' and an MLflow 'metric', and why might a lifecycle use both?
4	How would you prove to an auditor that a specific production model version is fully reproducible from source?

LAB 3

Locked, Containerized Environment Setup

Building fully reproducible, multi-stage Docker environments with locked dependencies

Objective	Eliminate 'works on my machine' drift by producing bit-for-bit reproducible Python environments using dependency locking, and packaging them into optimized, multi-stage Docker images for training vs. serving.
Tools	Docker, pip-tools / conda-lock, Python 3
Duration	90-120 minutes

3.1 Background

A basic environment setup lab installs packages from a loosely pinned **requirements.txt**. At an advanced level, reproducibility requires **fully locked** dependency graphs (exact versions and hashes for every transitive dependency), separation of training-time and serving-time dependencies (a training image needs pandas/matplotlib/jupyter; a serving image should not), and multi-stage Docker builds that keep the final runtime image minimal and free of build tooling.

3.2 Setup

```
pip install pip-tools
mkdir env_lab && cd env_lab
mkdir -p src
```

3.3 Separate and Lock Dependencies

Create two loose input files:

```
# requirements.in (training-time)
pandas
scikit-learn
mlflow
matplotlib

# requirements-serve.in (serving-time, minimal)
scikit-learn
fastapi
uvicorn
joblib
```

Compile fully pinned, hash-locked lockfiles:

```
pip-compile --generate-hashes --output-file=requirements.lock.txt requirements.in
pip-compile --generate-hashes --output-file=requirements-serve.lock.txt requirements-serve.in

# Install strictly from the lockfile (fails if anything is unpinned or hash-mismatched)
pip-sync requirements.lock.txt
```

Inspect the generated lockfile and note that every package, including transitive dependencies, is pinned to an exact version with a SHA-256 hash — this is what makes the environment bit-for-bit reproducible, not just 'roughly the same version'.

3.4 Multi-Stage Dockerfile — Training Image

```
# Dockerfile.train
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.lock.txt .
RUN pip install --no-cache-dir --require-hashes -r requirements.lock.txt --target=/deps

FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=builder /deps /usr/local/lib/python3.11/site-packages
COPY src/ src/
ENV PYTHONUNBUFFERED=1
ENTRYPOINT ["python", "src/train.py"]
```

3.5 Multi-Stage Dockerfile — Minimal Serving Image

```
# Dockerfile.serve
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements-serve.lock.txt .
RUN pip install --no-cache-dir --require-hashes -r requirements-serve.lock.txt --target=/deps

FROM gcr.io/distroless/python3-debian12 AS runtime
WORKDIR /app
COPY --from=builder /deps /usr/lib/python3.11/site-packages
COPY src/serve.py src/serve.py
COPY models/model.pkl models/model.pkl
USER 1000
ENTRYPOINT ["python", "src/serve.py"]
```

Using a distroless base for the serving image removes the shell, package manager, and any tooling not strictly required at runtime — reducing the attack surface, an important consideration in a regulated or security-conscious environment.

3.6 Build, Compare, and Verify

```
docker build -f Dockerfile.train -t ml-lab:train .
docker build -f Dockerfile.serve -t ml-lab:serve .

docker images ml-lab          # compare image sizes: train vs. serve
docker run --rm ml-lab:serve python -c "import sklearn; print(sklearn.__version__)"

# Reproducibility check: rebuild on a clean machine/CI runner and diff the installed package list
docker run --rm ml-lab:train pip freeze > freeze_run1.txt
docker run --rm ml-lab:train pip freeze > freeze_run2.txt
diff freeze_run1.txt freeze_run2.txt # should be empty
```

3.7 Exercises

- Add a **.dockerignore** and confirm build context size drops; explain why a large build context slows CI.
- Introduce a deliberate dependency conflict (e.g. two packages requiring incompatible numpy versions) and observe how **pip-compile** surfaces it at lock time rather than at deploy time.
- Add a CI step that fails the build if **requirements.lock.txt** is out of date relative to **requirements.in** (**pip-compile --dry-run**).
- Scan both images with a vulnerability scanner (e.g. **docker scout** or **trivy image ml-lab:serve**) and compare the finding count between the training and serving images.

#	Viva / Discussion Question
1	Why is a loosely pinned requirements.txt insufficient for production ML reproducibility?

2	What security and operational benefits come from separating training-time and serving-time dependencies?
3	Why use a multi-stage Docker build instead of installing everything directly in the final image?
4	How would you detect and prevent 'dependency drift' between what was tested in CI and what actually runs in production?

LAB 4

Model Serialization & Configuration Management

Safe, portable model serialization and structured, environment-aware configuration

Objective	Compare and implement multiple model serialization strategies (including cross-framework ONNX export) and replace scattered config values with a structured, composable configuration system using Hydra.
Tools	Python 3, scikit-learn, skl2onnx / onnxruntime, Hydra, OmegaConf
Duration	90-120 minutes

4.1 Background

A basic lab saves a model with **pickle** and calls it done. At an advanced level, serialization choices carry real trade-offs: **pickle/joblib** are fast but insecure to deserialize from an untrusted source and tightly coupled to the exact library version used to train; **ONNX** gives a framework-agnostic, versioned interchange format suited to serving in a different language/runtime than training; **PMML** and **safetensors** exist for other trade-off points. Likewise, configuration management should move from hardcoded constants or a flat YAML file to a composable, environment-aware system that supports overrides, validation, and command-line composition.

4.2 Setup

```
pip install scikit-learn skl2onnx onnxruntime hydra-core omegaconf joblib
mkdir serialization_config_lab && cd serialization_config_lab
mkdir -p src conf conf/model conf/data
```

4.3 Train and Serialize a Model Three Ways

```
import joblib
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_wine
from skl2onnx import to_onnx

X, y = load_wine(return_X_y=True)
model = RandomForestClassifier(n_estimators=100, random_state=42).fit(X, y)

# 1. joblib – fast, Python/sklearn-version-coupled, NOT safe to load from an untrusted source
joblib.dump(model, 'model.joblib')

# 2. ONNX – framework-agnostic, versioned, verifiable schema
onnx_model = to_onnx(model, X[:1].astype(np.float32))
with open('model.onnx', 'wb') as f:
    f.write(onnx_model.SerializeToString())

# 3. Plain-parameter export – fully inspectable, no code execution risk at all
import json
params_export = {
    'model_type': 'RandomForestClassifier',
    'sklearn_version': __import__('sklearn').__version__,
    'params': model.get_params(),
    'n_features': X.shape[1],
    'classes': model.classes_.tolist(),
```

```
}
json.dump(params_export, open('model_metadata.json', 'w'), indent=2)
```

4.4 Verify the ONNX Model is Portable and Equivalent

```
import onnxruntime as ort
import numpy as np

sess = ort.InferenceSession('model.onnx')
input_name = sess.get_inputs()[0].name
onnx_preds = sess.run(None, {input_name: X[:5].astype(np.float32)})[0]

sklearn_preds = model.predict(X[:5])
assert (onnx_preds == sklearn_preds).all(), 'ONNX output diverges from source model!'
print('ONNX predictions match the source scikit-learn model.')
```

4.5 Demonstrate the Pickle Security Risk (Conceptually)

Pickle deserialization executes arbitrary code embedded in the file. **Never** unpickle a model from an untrusted or unauthenticated source. Discuss and document (do not execute untrusted code) why this matters:

- A pickled object's `__reduce__` method can call arbitrary functions during **load()** — including OS commands.
- A model registry that accepts uploads from external or semi-trusted parties should reject raw pickle uploads, or scan/sandbox them, or require ONNX/safetensors instead.
- Mitigations: restrict model-loading to a trusted, access-controlled registry only; prefer ONNX/safetensors for any cross-boundary transfer; if pickle must be used internally, sign artifacts and verify signatures before loading.

4.6 Structured Configuration with Hydra

Replace a flat config with composable Hydra config groups:

```
# conf/config.yaml
defaults:
  - model: random_forest
  - data: wine
  - _self_

environment: dev
mlflow_tracking_uri: http://localhost:5000

# conf/model/random_forest.yaml
name: random_forest
n_estimators: 100
max_depth: 8
random_state: 42

# conf/model/gradient_boosting.yaml
name: gradient_boosting
n_estimators: 200
learning_rate: 0.05
max_depth: 3

# conf/data/wine.yaml
dataset: wine
test_size: 0.2
```

4.7 Consume the Config with Environment Overrides

```

import hydra
from omegaconf import DictConfig, OmegaConf

@hydra.main(version_base=None, config_path='conf', config_name='config')
def main(cfg: DictConfig) -> None:
    print(OmegaConf.to_yaml(cfg))
    if cfg.model.name == 'random_forest':
        from sklearn.ensemble import RandomForestClassifier as Model
        kwargs = {'n_estimators': cfg.model.n_estimators,
                  'max_depth': cfg.model.max_depth,
                  'random_state': cfg.model.random_state}
    else:
        from sklearn.ensemble import GradientBoostingClassifier as Model
        kwargs = {'n_estimators': cfg.model.n_estimators,
                  'learning_rate': cfg.model.learning_rate,
                  'max_depth': cfg.model.max_depth}
    model = Model(**kwargs)
    print(f'Instantiated {cfg.model.name} for environment={cfg.environment}')

if __name__ == '__main__':
    main()

```

```

python src/run.py                                # uses defaults (random_forest, dev)
python src/run.py model=gradient_boosting         # swap the entire model config group
python src/run.py environment=prod model.max_depth=12 # override a single nested value
python src/run.py --multirun model=random_forest,gradient_boosting # sweep across configs

```

4.8 Separate Secrets from Config

Configuration values (hyperparameters, dataset names) and secrets (API keys, database credentials) must never live in the same committed file. Demonstrate the separation:

```

# conf/config.yaml keeps only non-secret settings and a reference name:
secrets_provider: env # or 'vault', 'aws_secrets_manager', etc.

# secrets are resolved at runtime from environment variables / a secrets manager, never from Y
AML:
import os
mlflow_token = os.environ['MLFLOW_TRACKING_TOKEN'] # fails loudly if unset, never defaults s
ilently

```

4.9 Exercises

- Add a third model config group (e.g. logistic regression) and use **--multirun** to sweep across all three, logging each to MLflow with the config captured as run parameters.
- Write a schema (Hydra **structured config** / a **dataclass**) that validates **max_depth** is a positive integer and rejects the config at startup if violated, rather than failing deep inside training.
- Benchmark load time and file size for joblib vs. ONNX vs. the JSON metadata approach on a larger model, and summarize the trade-offs in a short table.
- Design (in writing, no need to fully implement) a model-registry upload policy that only accepts ONNX or signed artifacts, and explain how you would enforce it at the API layer.

4.10 Expected Outcome

Working joblib, ONNX, and metadata-only serializations of the same model with verified prediction equivalence between joblib and ONNX; a Hydra-based configuration system supporting config-group swaps, single-value overrides, and multirun sweeps; and a documented, enforced separation between configuration and secrets.

1	What specific security risk does unpickling a model from an untrusted source introduce, and how does ONNX avoid it?
2	Why is version coupling (e.g. a joblib file tied to an exact scikit-learn version) a production risk, and how would you detect it before it causes an outage?
3	How does Hydra's config composition differ from a single flat YAML file, and what does that enable operationally (e.g. multirun sweeps, environment overrides)?
4	Why must secrets never be stored in the same file or repository as non-sensitive configuration, even if the repository is private?

End of Lab Manual — Advanced MLOps Practice Labs (Introduction to MLOps, ML Lifecycle, Environment Setup, Model Serialization & Configuration Management).